

-----<SQL note>-----

Creating sql command{

-- Create Database

```
CREATE DATABASE IF NOT EXISTS varsity;
```

```
USE varsity;
```

-- Create Table

```
CREATE TABLE IF NOT EXISTS student (
```

```
    id INT PRIMARY KEY,
```

```
    name VARCHAR(50) NOT NULL,
```

```
    age INT NOT NULL
```

```
);
```

-- Insert Data

```
INSERT INTO student VALUES (1, 'Tanim', 21);
```

```
INSERT INTO student VALUES (2, 'Tanzim', 20);
```

```
INSERT INTO student VALUES (3, 'Rahim', 22);
```

```
-- Show All Data
```

```
SELECT * FROM student;
```

```
drop database if exists varsity;
```

```
}
```

SQL datatypes

DATATYPE	DESCRIPTION	USAGE
CHAR	string(0-255), can store characters of fixed length	CHAR(50)
VARCHAR	string(0-255), can store characters up to given length	VARCHAR(50)
BLOB	string(0-65535), can store binary large object	BLOB(1000)
INT	integer(-2,147,483,648 to 2,147,483,647)	INT
TINYINT	integer(-128 to 127)	TINYINT
BIGINT	integer(-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807)	BIGINT
BIT	can store x-bit values. x can range from 1 to 64	BIT(2)
FLOAT	Decimal number - with precision to 23 digits	FLOAT
DOUBLE	Decimal number - with 24 to 53 digits	DOUBLE
BOOLEAN	Boolean values 0 or 1	BOOLEAN
DATE	date in format of YYYY-MM-DD ranging from 1000-01-01 to 9999-12-31	DATE
YEAR	year in 4 digits format ranging from 1901 to 2155	YEAR

SQL Datatypes

Signed & Unsigned

TINYINT UNSIGNED (0 to 255)

TINYINT (-128 to 127)

numeric
↓
INT
FLOAT
DOUBLE
TINYINT

—ve +ve
(+ve)
ages
salary

Types of SQL Commands

DDL (Data Definition Language) : create, alter, rename, truncate & drop

DQL (Data Query Language) : select

DML (Data Manipulation Language) : , insert, update & delete

DCL (Data Control Language) : grant & revoke permission to users

TCL (Transaction Control Language) : start transaction, commit, rollback &

Database related Queries

CREATE DATABASE db_name;

CREATE DATABASE IF NOT EXISTS db_name;

CREATE DATABASE IF NOT EXISTS college;

DROP DATABASE db_name;

DROP DATABASE IF EXISTS db_name;

SHOW DATABASES;

SHOW TABLES;



(To activate Connection)

services.msc → write this to command and then select ur server

Constraints

SQL constraints are used to specify rules for data in a table.

NOT NULL

columns cannot have a null value

```
col1 int NOT NULL
```

UNIQUE

all values in column are different

```
col2 int UNIQUE
```

PRIMARY KEY

makes a column unique & not null but used only for one

```
id int PRIMARY KEY
```

```
CREATE TABLE temp (  
  id int not null,  
  PRIMARY KEY (id)  
);
```

straints >

Constraints

FOREIGN KEY prevent actions that would destroy links between tables

```
CREATE TABLE temp (  
  cust_id int,  
  FOREIGN KEY (cust_id) references customer(id)  
);
```

DEFAULT sets the default value of a column

salary **INT DEFAULT** 25000

id	salary	

def
value

constraints >

Constraints

CHECK it can limit the values allowed in a column

```
CREATE TABLE city (  
  id INT PRIMARY KEY,  
  city VARCHAR(50),  
  age INT,  
  CONSTRAINT age_check CHECK (age >= 18 AND city="Delhi")  
);
```

```
CREATE TABLE newTab (  
  age INT CHECK (age >= 18)  
);
```

city

id	city	int



```
• SELECT DISTINCT city FROM student;
```

If we use DISTINCT then it means the duplicate data will not come from the table for example in this ss we can see that DISTINCT constrain has been used means the duplicate data from column “city” will not come.

Where Clause:

Where Clause

To define some conditions

```
SELECT col1, col2 FROM table_name  
WHERE conditions;
```

```
SELECT * FROM student WHERE marks > 80;  
SELECT * FROM student WHERE city = "Mumbai";
```


OPERATORS:

Operators

AND (to check for both conditions to be true)

```
SELECT * FROM student WHERE marks > 80 AND city = "Mumbai";
```

OR (to check for one of the conditions to be true)

```
SELECT * FROM student WHERE marks > 90 OR city = "Mumbai";
```

Operators

Between (selects for a given range)

```
SELECT * FROM student WHERE marks BETWEEN 80 AND 90;
```

Handwritten red arrows point to 80 and 90, with the word "inclusive" written in red above them.

In (matches any value in the list)

```
SELECT * FROM student WHERE city IN ("Delhi", "Mumbai");
```

NOT (to negate the given condition)

```
SELECT * FROM student WHERE city NOT IN ("Delhi", "Mumbai");
```

Handwritten red dashed line with a dot below the query.

LIMIT Clause:

Limit Clause

Sets an upper limit on number of (tuples)rows to be returned

```
SELECT * FROM student LIMIT 3;
```

```
SELECT col1, col2 FROM table_name  
LIMIT number;
```

Order by CLAUSE:

Order By Clause

To sort in ascending (ASC) or descending order (DESC)

```
SELECT * FROM student  
ORDER BY city ASC;
```

```
SELECT col1, col2 FROM table_name  
ORDER BY col_name(s) ASC;
```

1 2 3 4 5 → ASCENDING
5 4 3 2 1 → DESCENDING

A ✓
B ✓
C ✓
:
:
:
:
:



Aggregate Function:

Aggregate Functions

Aggregate functions perform a calculation on a set of values, and return a single value.

- COUNT() *~ qum*
- MAX() *~ shradha*
- MIN() *~ value*
- SUM()
- AVG()

Get Maximum Marks

```
SELECT max(marks)
FROM student;
```

Get Average marks

```
SELECT avg(marks)
FROM student;
```

Aggregate Functions >



Group By Clause:

Group By Clause

Groups rows that have the same values into summary rows.
It collects data from multiple records and groups the result by one or more column.

*Generally we use group by with some *aggregation function*.

Count number of students in each city

```
SELECT city, count(name)
FROM student
GROUP BY city;
```

2) De
3) M



Exp Code:

```
create database collage;
use collage;
create table student(
roll int,
name varchar(50),
marks int,
grade float,
city varchar(50)
);
INSERT INTO student (roll, name, marks, grade, city) VALUES
(1, "Tanim", 100, 4.00, "Barishal"),
(2, "Sumon", 900, 4.50, "Dhaka"),
(3, "Labib", 1040, 6.00, "Newyork"),
(4, "Tanzim", 200, 5.00, "Barishal"),
(5, "Rafi", 780, 3.75, "Chittagong"),
(6, "Ayesha", 920, 4.20, "Sylhet"),
(7, "Rumana", 650, 3.50, "Comilla"),
(8, "Fahim", 830, 3.90, "Mymensingh"),
(9, "Nashit", 970, 4.30, "Khulna"),
(10, "Sadia", 710, 3.60, "Jessore");

select city,count(roll) from student
group by city;
```

Output:

Result Grid				 Filter Rows:
	city	count(roll)		
▶	Barishal	2		
	Dhaka	1		
	Newyork	1		
	Chittagong	1		
	Sylhet	1		
	Comilla	1		
	Mymensingh	1		
	Khulna	1		
	Jessore	1		

select city,avg(marks) from student group by city order by city;

	city	avg(marks)
▶	Barishal	150.0000
	Chittagong	780.0000
	Comilla	650.0000
	Dhaka	900.0000
	Jessore	710.0000
	Khulna	970.0000
	Mymensingh	830.0000
	Newyork	1040.0000
	Sylhet	920.0000

Having Clause:

Having Clause

Similar to Where i.e. applies some condition on rows.

Used when we want to apply any **condition after grouping**.

Count number of students in each city where max marks cross 90.

```
SELECT count(name), city
FROM student
GROUP BY city
HAVING max(marks) > 90;
```

General Order:

General Order

```
SELECT column(s)
FROM table_name
WHERE condition
GROUP BY column(s)
HAVING condition
ORDER BY column(s) ASC;
```

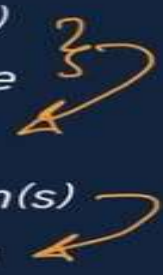


Table related Queries:

Table related Queries

Update (to update existing rows)

```
UPDATE table_name  
SET col1 = val1, col2 = val2  
WHERE condition;
```

```
UPDATE student  
SET grade = "0"  
WHERE grade = "A";
```

```
SET SQL_SAFE_UPDATES = 0;
```

Here this is used to turn off the safe mode so that user can update there data

Foreign Keys:

Revisiting FK

Foreign Key (dept-id) REFERENCES dept

dept	id	name
(PK)	101	Science
	102	English
	103	Hindi

↓ parent table

teacher	id	name	dept-id
	101	Adam	101
	102	Bob	103
	103	Casey	102
	104	Donald	102

↓ child table

FK



Cascading in FK:

Cascading for FK

dept ↔ teacher

Delete
On Update Cascade

When we create a foreign key using this option, it deletes the referencing rows in the child table when the referenced row is deleted in the parent table which has a primary key.

Update
On Delete Cascade

When we create a foreign key using UPDATE CASCADE the referencing rows are updated in the child table when the referenced row is updated in the parent table which has a primary key.

```
CREATE TABLE student (  
  id INT PRIMARY KEY,  
  courseID INT,  
  FOREIGN KEY(courseID) REFERENCES course(id)  
  ON DELETE CASCADE  
  ON UPDATE CASCADE
```

scading Foreign Keys >



Table Related Queries:

Table related Queries

Alter (to change the schema)

↳ design (columns, datatype, constraints)

ADD Column

ALTER TABLE *table_name*

ADD COLUMN *column_name* *datatype* *constraint*;

DROP Column

ALTER TABLE *table_name*

DROP COLUMN *column_name*;

RENAME Table

ALTER TABLE *table_name*

RENAME TO *new_table_name*;

Command >

Table related Queries

Truncate (to delete table's data)

TRUNCATE TABLE *table_name* ;

↓
DROP
↓
delete table

↓
TRUNCATE
↓
table data

```
UPDATE student  
SET grade = "0"  
WHERE grade = "A";
```



SQL CHEAT SHEET

Brought to you by programmingvalley.com

Basic SQL Commands	Joins	Transactions	Window Functions
• SELECT • INSERT INTO • UPDATE • DELETE	• INNER JOIN • LEFT JOIN (or LEFT OUTER JOIN) • RIGHT JOIN (or RIGHT OUTER JOIN) • FULL JOIN (or FULL OUTER JOIN) • CROSS JOIN • SELF JOIN	• BEGIN TRANSACTION • COMMIT • ROLLBACK • SAVEPOINT • SET TRANSACTION ISOLATION LEVEL	• ROW_NUMBER() • RANK() • DENSE_RANK() • NTILE() • LEAD() • LAG() • FIRST_VALUE() • LAST_VALUE() • SUM() OVER() • AVG() OVER() • PARTITION BY • ORDER BY
Filtering Data	Subqueries	Views	Set Operations
• WHERE • AND, OR, NOT • IN • BETWEEN • LIKE • IS NULL, IS NOT NULL • EXISTS	• Subqueries in SELECT • Subqueries in WHERE • Subqueries in FROM	• CREATE VIEW • DROP VIEW • ALTER VIEW • INSERT INTO VIEW • UPDATE VIEW	• UNION • UNION ALL • INTERSECT • EXCEPT
Date and Time Functions	Sorting and Limiting Data	Triggers	Aggregating Data
• GETDATE() • CURRENT_TIMESTAMP • DATEADD() • DATEDIFF() • DATEPART() • DATE_FORMAT() • NOW() • EXTRACT() • TIMESTAMPDIFF()	• ORDER BY • LIMIT • DISTINCT	• CREATE TRIGGER • DROP TRIGGER • AFTER INSERT/UPDATE/DELETE • BEFORE INSERT/UPDATE/DELETE	• COUNT() • SUM() • AVG() • MIN() • MAX() • GROUP BY • HAVING
Data Modification	Common Table Exp.	Indexing	
• UPDATE	• WITH CTE AS (...) • RECURSIVE CTE • WITH TEMPORARY CTE	• CREATE INDEX • DROP INDEX • UNIQUE INDEX • FULL-TEXT INDEX • COMPOSITE INDEX	



Programming Valley

Follow

Alan Avry · Celebrate Conditional Logic (prod. by d.higgs) (Instrumental)

SQL Cheat Sheet

See more

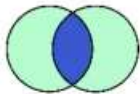


SQL JOINS EXPLAINED

Brought to you by programmingvalley.com

INNER JOIN

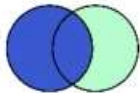
Returns only matching records from both tables.



```
SELECT *  
FROM A  
INNER JOIN B ON A.key = B.key
```

LEFT JOIN

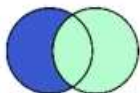
Returns all records from A, and matching ones from B (if any).



```
SELECT *  
FROM A  
LEFT JOIN B ON A.key = B.key
```

LEFT ANTI JOIN

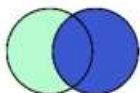
Returns records from A that have no match in B.



```
SELECT *  
FROM A  
LEFT JOIN B ON A.key = B.key WHERE B.key IS NULL
```

RIGHT JOIN

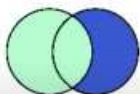
Returns all records from B, and matching ones from A (if any).



```
SELECT *  
FROM A  
RIGHT JOIN B ON A.key = B.key
```

RIGHT ANTI JOIN

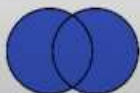
Returns records from B that have no match in A.



```
SELECT *  
FROM A  
RIGHT JOIN B ON A.key = B.key WHERE A.key IS NULL
```

FULL JOIN

Returns all records from A and B, matched where possible.



```
SELECT *  
FROM A  
FULL JOIN B ON A.key = B.key
```



Programming Valley · Follow

FULL ANTI JOIN

Returns all records from A and B that do not match each other.

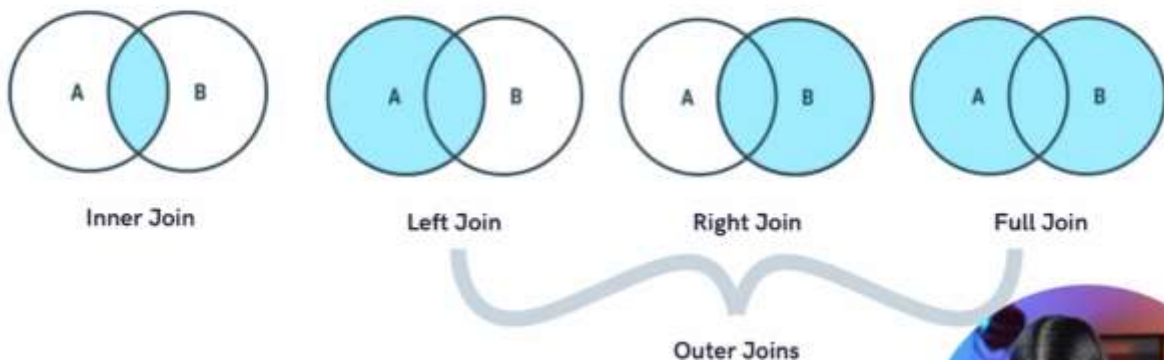


```
SELECT *  
FROM A FULL JOIN B ON A.key = B.key  
WHERE A.key IS NULL OR B.key IS NULL
```

Save this
... See more

JOINS

Types of Joins (Venn Di)



Inner Join

Returns records that have matching values in both tables

Syntax

SELECT *column(s)*

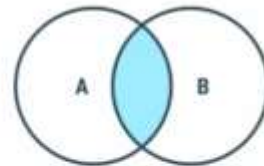
table A

table B

FROM *tableA*

INNER JOIN *tableB*

ON *tableA.col_name = tableB.col_name;*



Inner Join

Example

student

student_id	name
101	adam
102	bob
103	casey

course

student_id	course
102	english
105	math
103	science
107	computer science

```
SELECT *  
FROM student  
INNER JOIN course  
ON student.student_id = course.student_id;
```

Result

student_id	name	course
102	bob	english
103	casey	science

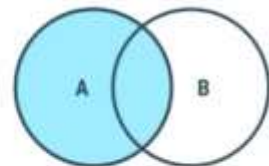


Left Join

Returns all records from the left table, and the matched records from the right table

Syntax

```
SELECT column(s)  
FROM tableA  
LEFT JOIN tableB  
ON tableA.col_name = tableB.col_name;
```



Left Join

Example

student (left)

student_id	name
101 ✓	adam
102 ✓	bob
103 ✓	casey

course (right)

student_id	course
102	english
105	math
103	science
107	computer science

```
SELECT *  
FROM student as s  
LEFT JOIN course as c  
ON s.student_id = c.student_id;
```

Result

student_id	name	course
101 ✓	adam	null
102 ✓	bob	english
103 ✓	casey	science



Right Join

Example

student

student_id	name
101	adam
102	bob
103	casey

course

student_id	course
102	english
105	math
103	science
107	computer science

```
SELECT *  
FROM student as s  
RIGHT JOIN course as c  
ON s.student_id = c.student_id;
```

Result

student_id	course	name
102	english	bob
105	math	null
103	science	casey
107	computer science	null



Full Join

Returns all records when there is a match in either left or right table

Syntax in MySQL

```
SELECT * FROM student as a
LEFT JOIN course as b
ON a.id = b.id
UNION
SELECT * FROM student as a
RIGHT JOIN course as b
ON a.id = b.id;
```

LEFT JOIN
UNION
RIGHT JOIN



Full Join

Example

student

student_id	name
101	adam
102	bob
103	casey

course

student_id	course
102	english
105	math
103	science
107	computer science

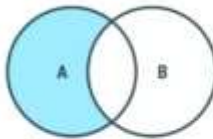
Result

student_id	name	course
101	adam	null
102	bob	english
103	casey	science
105	null	math
107	null	computer science

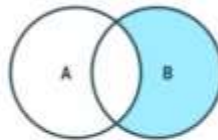
Think & Ans



Qs: Write SQL commands to display the right exclusive join :



Left Exclusive Join



Right Exclusive Join



```
SELECT *  
FROM student as a  
LEFT JOIN course as b  
ON a.id = b.id  
WHERE b.id IS NULL;
```

